

Machine Learning

Regularization



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

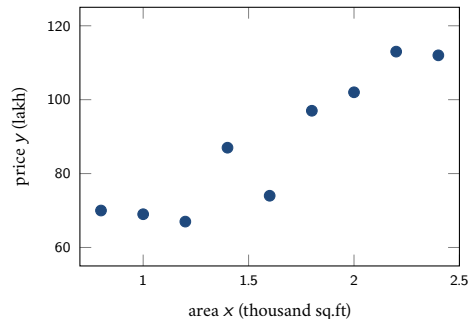
Session Outcome

- Understand why a model with many features fits the training data well but fails on unseen data, and control this using *Ridge*, *Lasso* and *Elastic Net* regularisation.

Regularization

The Dataset: Nine Houses, One Feature

House	Area $\times$ (k sq.ft)	Price y (lakh)
1	0.8	70
2	1.0	69
3	1.2	67
4	1.4	87
5	1.6	74
6	1.8	97
7	2.0	102
8	2.2	113
9	2.4	112



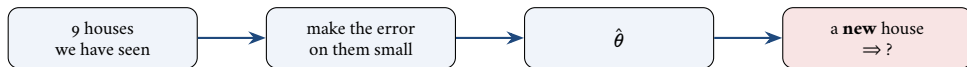
Observations

- Bigger flats cost more
- The trend is clearly rising and roughly straight.

Minimizing MSE

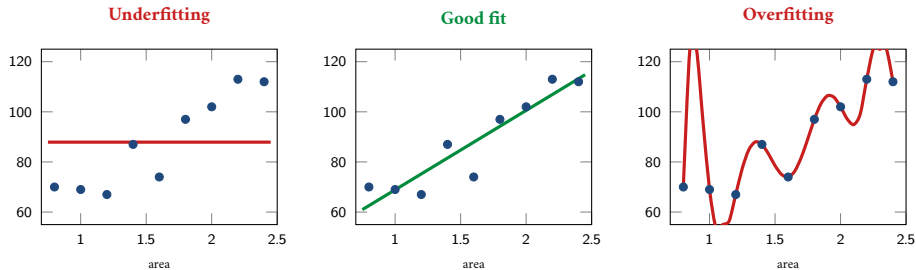
Every model so far was fitted by making the training error as small as possible:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left(\theta^\top \mathbf{x}^{(i)} - y^{(i)} \right)^2 \quad \Rightarrow \quad \hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$



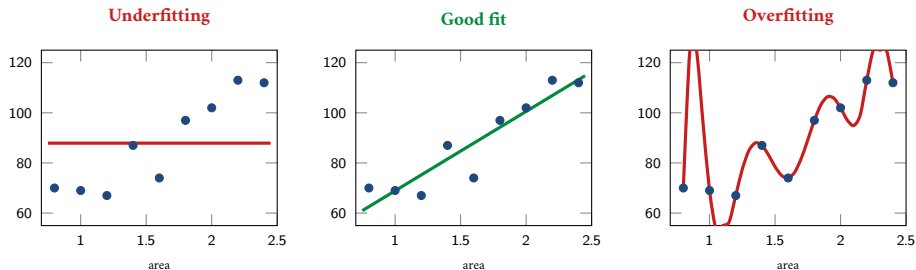
- No conditions in $J(\theta)$ to bound θ
- Is a smaller training error always a better model?
- Let us check with new data.

Three Models for the Same Dataset



For the same dataset we have three different models

Three Models for the Same Dataset



For the same dataset we have three different models

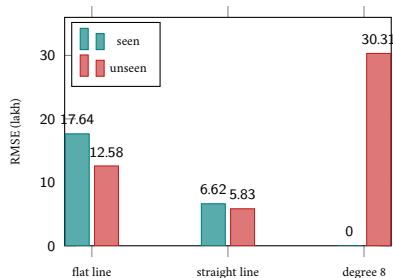
- **Underfitting:** one flat price for everybody!! (Simple, but misses the trend)
- **Good fit:** follows the trend.
- **Overfitting:** passes through *every* point exactly.
 - It even dives below near $x = 1.1$
 - Shoots above 125 lakh near $x = 0.85$

RMSE for Underfitting Vs Goodfit Vs Overfitting

Model	RMSE on the 9 seen houses	RMSE on the 8 unseen
flat line (underfit)	17.64	12.58
straight line (good)	6.62	5.83
degree-8 curve (overfit)	0.00	30.31

Observation

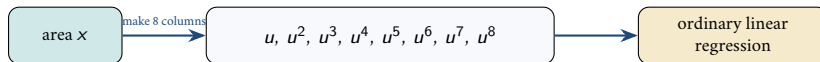
The overfitted model has the best training score with zero error and a perfect fit. It is also the worst model of the three for testing data.



Overfit Model: Using Regression

We have only one measurement, area x . To let the model bend, we use higher degree polynomial:

$$\hat{y} = \theta_0 + \theta_1 u + \theta_2 u^2 + \theta_3 u^3 + \cdots + \theta_8 u^8, \quad u = \frac{x - 1.6}{0.8}$$

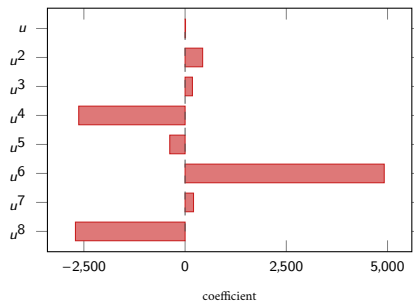


Why rescale x to u ?

With x in thousands, x^8 would range over four orders of magnitude. Bring the range to $[-1, 1]$, so every column has a comparable size. *This is standardisation*

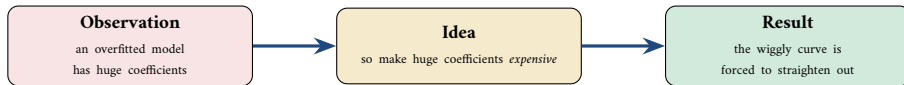
Coefficients of the Overfitted Model

Term	Coefficient θ_i
θ_0 (intercept)	74.00
$\theta_1 u$	10.10
$\theta_2 u^2$	433.62
$\theta_3 u^3$	181.16
$\theta_4 u^4$	-2626.76
$\theta_5 u^5$	-378.31
$\theta_6 u^6$	+4918.04
$\theta_7 u^7$	208.05
$\theta_8 u^8$	-2707.91
$\sum_j \theta_j $	11 464



Overfitting is not a mysterious idea: it is visibly large numbers in θ .

Regularization



$$J(\theta) = \underbrace{\frac{1}{m} \|\mathbf{X}\theta - \mathbf{y}\|^2}_{\text{fit the 9 houses}} + \lambda \underbrace{\text{pen}(\theta)}_{\text{keep } \theta \text{ small}}$$

- Three choices of $\text{pen}(\theta)$ give three famous methods:

Ridge	$\lambda \sum_j \theta_j^2$	squares — shrinks everything smoothly
Lasso	$\lambda \sum_j \theta_j $	absolute values — deletes some terms outright
Elastic Net	mixture of both	shrinks <i>and</i> deletes

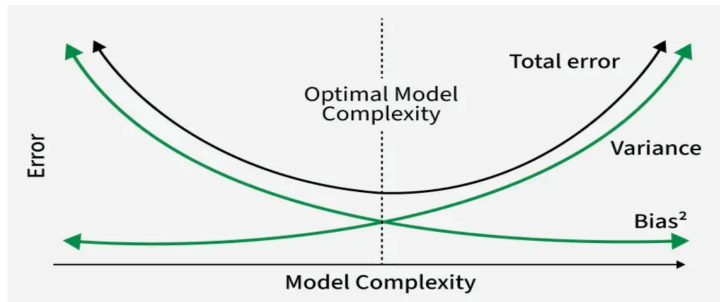
Q. How to pick λ for each method?

A: Choose λ which minimizes the Total Error = Bias² + Variance + Noise

Bias and Variance

Bias and Variance are two types of prediction errors

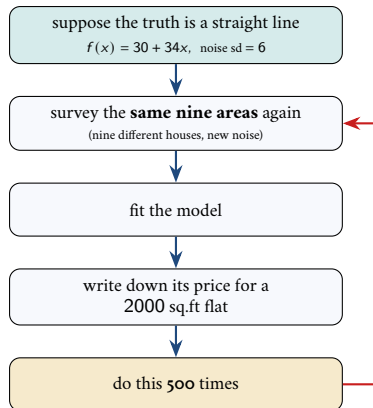
- **Bias** refers to the error caused by making wrong or simple assumptions
 - Leads to **underfitting**
- **Variance** refers to the error caused by making the model too sensitive to training data.
 - Leads to **overfitting**



Source: GeekforGeeks

Figure: Bias-Variance Tradeoff

Experiment to Reveal Bias and Variance of a Model



The two questions we can now answer

Bias: is the *average* of the 500 answers right?

Variance: how much do the 500 answers disagree with each other?

- True price of that flat: $f(2.0) = \mathbf{98.0}$ lakh.

Redraw the Fit

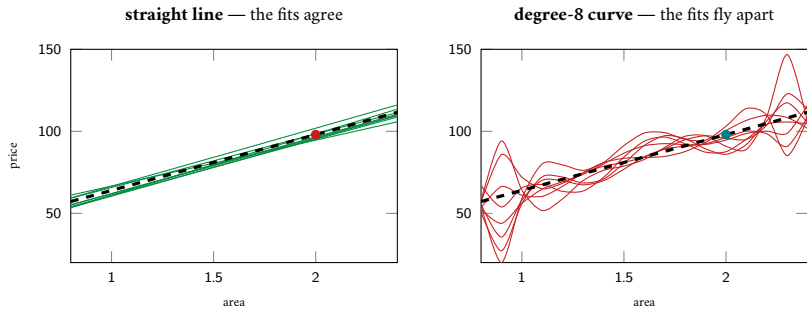
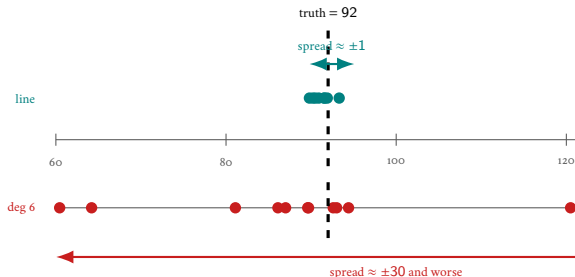


Figure: Dashed black = the truth. Each thin line is one repeat of the survey.

Left: change the houses, get almost the same line. Right: change the houses, get a completely different curve.



The Same Eight Fits Seen for One Sample House



Measured at Area = 2.0 over 500 repeats

	line	deg 6
average prediction	91.51	93.06
truth	92.00	92.00
bias	-0.49	+1.06
Bias ²	0.24	1.13
Variance	2.07	229.16

$$\bar{y} = \frac{1}{S} \sum_{s=1}^S \hat{y}^{(s)}$$

$$\text{Bias} = \bar{y} - f(x^*)$$

$$\text{Bias}^2 = (\bar{y} - f(x^*))^2$$

$$\text{Variance} = \frac{1}{S} \sum_{s=1}^S (\hat{y}^{(s)} - \bar{y})^2$$

Ridge Regression

- A regression model that uses the L_2 regularization technique is called Ridge regression.
- It adds the squared magnitude of the coefficient as a penalty term to the loss function.

$$\text{Cost} = J(\theta, \lambda) = \frac{1}{n} \|\mathbf{X}\theta - \mathbf{y}\|^2 + \lambda \sum_{j=1}^m \theta_j^2 \quad (1)$$

where

- n is number of data points
- m is number of features
- $\mathbf{y}$ is the vector containing actual target values
- $\hat{\mathbf{y}} = \mathbf{X}\theta$ is the vector containing the predicted target values
- λ is the regularization parameter

Ridge Regression has a closed form, exactly like ordinary least squares

Ridge Regression

$$J(\theta) = \|\mathbf{X}\theta - \mathbf{y}\|^2 + \lambda \sum_{j=1}^8 \theta_j^2$$

Derivation. Set the gradient to zero, using $\nabla_{\theta}(\theta^T \theta) = 2\theta$:

$$2\mathbf{X}^T(\mathbf{X}\theta - \mathbf{y}) + 2\lambda\theta = \mathbf{0} \implies (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\theta = \mathbf{X}^T\mathbf{y} \implies \hat{\theta}_{\text{ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

Effect of λ on Eigen values

Every eigenvalue $\mu_i \geq 0$ becomes $\mu_i + \lambda > 0$, so $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ is **always invertible**. Ridge **repairs** the normal equation.

Why the name “ridge”

A “ridge” of height λ is added along the diagonal:

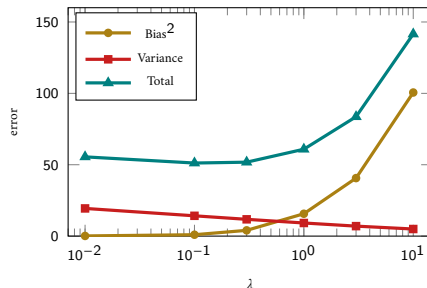
$$\begin{pmatrix} \square + \lambda & & \\ & \square + \lambda & \\ & & \square + \lambda \end{pmatrix}$$

Ridge Regression on the Overfit model

Total Error = Bias² + Variance + Noise

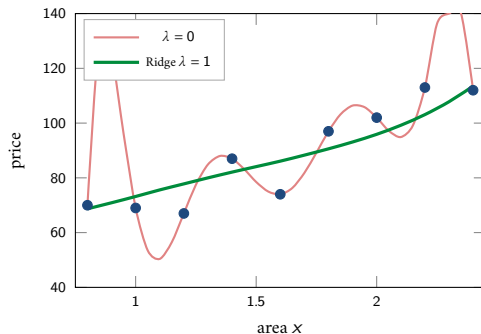
Same degree-8 model, same 500 surveys — but fitted with a penalty $\lambda \sum_j \theta_j^2$.

λ	Bias ²	Variance	Noise	Total
0	0.10	103.12	36	139.21
0.01	0.12	19.39	36	55.51
0.1	0.97	14.22	36	51.19
0.3	4.03	11.76	36	51.79
1	15.66	9.19	36	60.86
3	40.64	6.99	36	83.63
10	100.59	4.97	36	141.56



$\lambda = 0.1$ is the best choice for Ridge Regression in the example considered

Ridge on the Overfit (Degree-8) Model

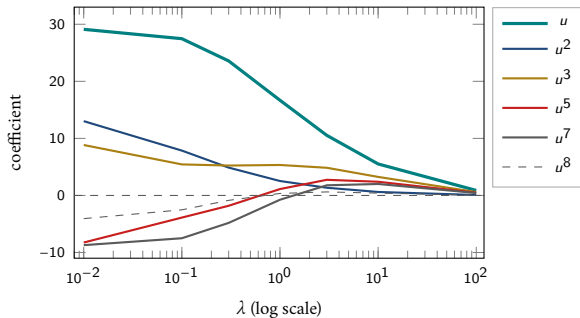


Term	$\lambda = 0$	Ridge $\lambda = 1$
θ_0	74.00	86.14
u	10.10	16.68
u^2	433.62	2.53
u^3	181.16	5.34
u^4	-2626.76	1.35
u^5	-378.31	1.12
u^6	4918.04	0.74
u^7	208.05	-0.75
u^8	-2707.91	0.35
$\sum_j \theta_j $	11 464	28.8
train RMSE	0.00	7.36
test RMSE	30.31	4.47

What the penalty fixed

All eight powers are **still there** — none was deleted — but every one has been squeezed to single digits. The curve straightens out on its own, and the error on unseen houses drops from 30.31 to **4.47**: better than the straight line we drew by hand.

Change in Coefficients with increase in λ



- Every curve slides **smoothly** towards zero.
- Not one of them ever **reaches** zero — look at $\lambda = 100$: the smallest is still 0.06, not 0.
- So Ridge always hands you a model that uses **all eight** powers, however tiny their weights.

Ridge in four words

It shrinks, never deletes.

Lasso Regression

A regression model which uses the L_1 Regularization technique is called LASSO (Least Absolute Shrinkage and Selection Operator) regression

- It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function
- This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$J = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \lambda \sum_{j=1}^m |\theta_j|$$

where

- n is number of data points
- m is number of features
- $\mathbf{y}$ is the vector containing actual target values
- $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$ is the vector containing the predicted target values
- λ is the regularization parameter

Coefficients in Lasso Vs Ridge

Ridge

$$J = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \lambda \sum_j \theta_j^2$$

$$\hat{\theta}_j^{\text{ridge}} = \frac{c_j}{d_j + \lambda},$$

$$\hat{\theta}_j^{\text{lasso}} = \frac{\text{sign}(c_j) \max\left(|c_j| - \frac{\lambda}{2}, 0\right)}{d_j},$$

Lasso

$$J = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \lambda \sum_j |\theta_j|$$

$$c_j = (\mathbf{X}^T \mathbf{y})_j$$

with $c_1 = 95.00$, $d_1 = 3.750$ and $c_2 = 8.54$, $d_2 = 2.766$.

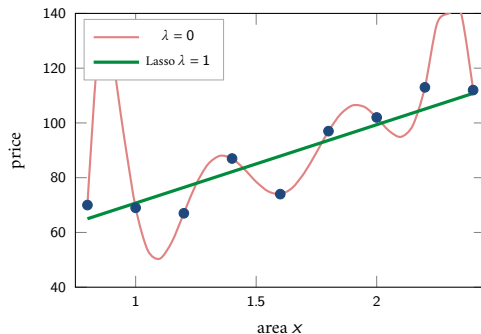
Ridge

λ	$\theta_1 (u)$	$\theta_2 (u^2)$
0	25.33	3.09
1	20.00	2.27
3	14.07	1.48
10	6.91	0.67
30	2.82	0.26

Lasso

λ	$\theta_1 (u)$	$\theta_2 (u^2)$
0	25.33	3.09
10	24.00	1.28
20	22.67	0.00
40	20.00	0.00
400	0.00	0.00

Lasso on the Overfit (Degree-8) Model

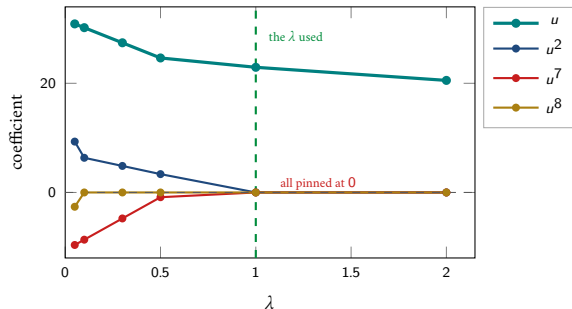


Term	$\lambda = 0$	Lasso $\lambda = 1$
θ_0	74.00	87.89
u	10.10	22.93
u^2	433.62	0
u^3	181.16	0
u^4	-2626.76	0
u^5	-378.31	0
u^6	4918.04	0
u^7	208.05	0
u^8	-2707.91	0
terms used	8	1
train RMSE	0.00	6.80
test RMSE	30.31	5.27

Lasso reduces the Model to a straight line

Given eight powers in the model, Lasso removes seven of them and keeps only u . The green curve of the Lasso Model is a line.

Lasso: Coefficients Vs λ



λ	alive	switched off here
0	8	—
0.05	4	u^3, u^4, u^5, u^6
0.1	3	u^8
0.5	3	—
1	1	u^2, u^7
4	1	—

- Once a curve touches zero it **stays** there.
- The weakest terms become zero first– the order is a ranking of usefulness.
- The last survivor is u : the one term the data really supports.

Elastic Net Regression

Elastic Net Regression is a combination of both L_1 as well as L_2 regularization.

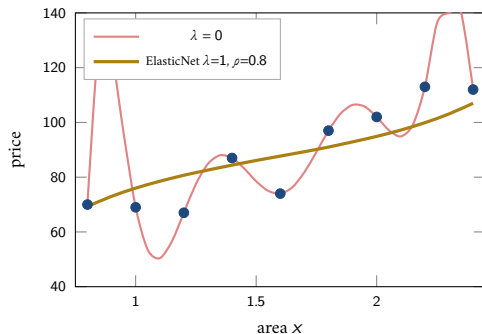
- It combines both L_1 (absolute values) and L_2 (squared values) penalties on the coefficients.
- Hyperparameter controls the ratio of the L_1 and L_2 regularization.

$$J(\theta) = \frac{1}{n} \|\mathbf{X}\theta - \mathbf{y}\|^2 + \lambda \left[\underbrace{\rho \sum_j |\theta_j|}_{\text{deletes}} + (1 - \rho) \underbrace{\sum_j \theta_j^2}_{\text{steadies}} \right], \quad \rho \in [0, 1]$$

where

- n is number of data points
- m is number of features
- $\mathbf{y}$ is the vector containing actual target values
- $\hat{\mathbf{y}} = \mathbf{X}\theta$ is the vector containing the predicted target values
- λ is the regularization parameter
- ρ is the hyperparameter
- $\rho = 1$ gives pure Lasso; $\rho = 0$ gives pure Ridge.
- The L_1 part still produces **exact zeros**.
- The L_2 part makes the problem **strictly convex**, s
- Two Parameters (knobs): λ (how hard) and ρ (which mixture). Tune both parameters on validation data.

Elastic Net on the Overfit (Degree-8) Model

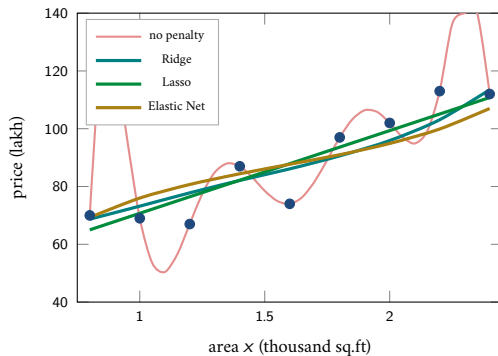


Term	$\lambda = 0$	ElasticNet
θ_0	74.00	87.70
u	10.10	12.97
u^2	433.62	0.44
u^3	181.16	4.54
u^4	-2626.76	0.02
u^5	-378.31	1.37
u^6	4918.04	0
u^7	208.05	0
u^8	-2707.91	0
terms used	8	5
train RMSE	0.00	8.90
test RMSE	30.31	5.22

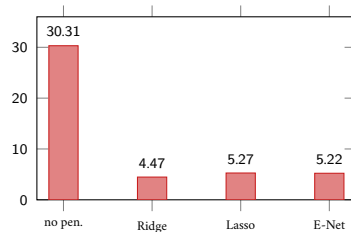
The Combination

Three powers are deleted outright (like Lasso) and the surviving odd powers are u, u^3, u^5 . the correlated group have same-sign weights (like Ridge).

All Four Models



RMSE on the 8 unseen houses



Model	terms	train	test
no penalty	8	0.00	30.31
Ridge $\lambda=1$	8	7.36	4.47
Lasso $\lambda=1$	1	6.80	5.27
E-Net $\lambda=1, \rho=0.8$	5	8.90	5.22

The model with the **best** training score is the **worst** on new houses. Every penalty cut the real error by about **6x**.

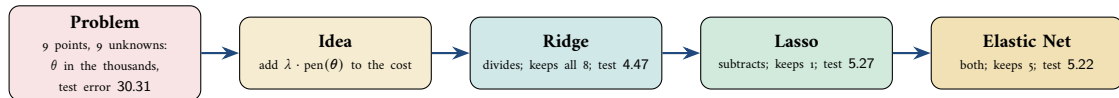
Ridge vs Lasso vs Elastic Net

	Ridge (L_2)	Lasso (L_1)	Elastic Net
Penalty	$\lambda \sum \theta_j^2$	$\lambda \sum \theta_j $	$\lambda [\rho \sum \theta_j + (1 - \rho) \sum \theta_j^2]$
Reaches exactly 0	never	yes	yes
Feature selection	no	yes	yes
Closed form	yes	no	no
Solver	linear algebra	coordinate descent	coordinate descent
Correlated columns	shares between them	picks one at random	keeps the group
Knobs	λ	λ	λ and ρ
scikit-learn	Ridge(alpha)	Lasso(alpha)	ElasticNet(alpha, l1_ratio)
On our 9 houses	8 terms, test 4.47	1 term, test 5.27	5 terms, test 5.22
Choose it when	every column helps a bit	most columns are useless	correlated groups <i>and</i> junk

A decision rule you can actually use

Start with **Ridge** — it is the safest and has a closed form. Move to **Lasso** when you must *explain* the model or you suspect most columns are noise. Move to **Elastic Net** when Lasso's chosen list keeps changing every time you re-run it.

Summary — Regularisation



- With as many unknowns as data points a model can fit perfectly and still be useless. The visible symptom is coefficients in the thousands with alternating signs.
- **Ridge** $\lambda \sum \theta_j^2$: closed form $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$, always invertible, **divides** each coefficient — shrinks but never deletes.
- **Lasso** $\lambda \sum |\theta_j|$: no closed form, weak terms hit **exactly zero**, so it selects features.
- **Elastic Net** mixes both: deletes junk *and* keeps correlated groups together, giving a stable list.

Appendix: Implementation

A1. The Nine Houses and the Eight Powers

Running the code in this appendix reproduces **every number** on the Regularisation slides.

```

1 import numpy as np
2 from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
3
4 # the nine houses we are allowed to learn from
5 x = np.array([0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4])      # k sq.ft
6 y = np.array([70., 69., 67., 87., 74., 97., 102., 113., 112.])  # lakh
7
8 # eight unseen houses, at the areas in between
9 xt = np.array([0.9, 1.1, 1.3, 1.5, 1.7, 1.9, 2.1, 2.3])
10 yt = np.round(30 + 34*xt + np.random.default_rng(1227).normal(0, 6, 8), 1)
11
12 u, ut = (x - 1.6)/0.8, (xt - 1.6)/0.8      # rescale area to about [-1, 1]
13 P = np.column_stack([u**k for k in range(1, 9)])  # u, u^2, ..., u^8
14 Pt = np.column_stack([ut**k for k in range(1, 9)])
15
16 rmse = lambda m, A, b: float(np.sqrt(np.mean((m.predict(A) - b)**2)))
  
```

A2. The Four Fits

```

1 def report(tag, model):
2     model.fit(P, y)
3     print(f"{tag:18s} "
4           f"train={rmse(model,P,y):5.2f} "
5           f"test={rmse(model,Pt,yt):6.2f} "
6           f"terms={int((abs(model.coef_)>1e-8).sum())}")
7     print("    ", np.round(model.coef_, 2))
8
9 report("no penalty", LinearRegression())
10 report("Ridge lam=1", Ridge(alpha=1))
11 report("Lasso lam=1", Lasso(alpha=1, max_iter=500000))
12 report("ElasticNet",
13       ElasticNet(alpha=1, l1_ratio=0.8, max_iter=500000))

```

```

no penalty  train= 0.00 test= 30.31 terms=8
[  10.10  433.62  181.16 -2626.76
 -378.31 4918.04  208.05 -2707.91]

Ridge lam=1  train= 7.36 test=  4.47 terms=8
[16.68  2.53  5.34  1.35
  1.12  0.74 -0.75  0.35]

Lasso lam=1  train= 6.80 test=  5.27 terms=1
[22.93  0.    0.    0.
  0.    0.    0.    0. ]

ElasticNet   train= 8.90 test=  5.22 terms=5
[12.97  0.44  4.54  0.02
  1.37  0.    0.    0. ]

```


A3. Ridge and Lasso

Because $u = -1, -0.75, \dots, 1$ is symmetric, the columns u and u^2 are orthogonal and $\mathbf{X}^\top \mathbf{X}$ is diagonal — so each coefficient can be solved on its own.

```

1 A = np.column_stack([u, u**2])
2 c = A.T @ (y - y.mean())      # X'y -> [95.00, 8.54]
3 d = np.diag(A.T @ A)         # X'X diagonal -> [3.750, 2.766]
4
5 for lam in [0, 1, 3, 10, 30]:  # RIDGE: divide
6     print(lam, np.round(c / (d + lam), 2))
7
8 for lam in [0, 10, 20, 40]:    # LASSO: subtract
9     soft = np.sign(c) * np.maximum(np.abs(c) - lam/2, 0)
10    print(lam, np.round(soft / d, 2))

```

RIDGE	LASSO
0 [25.33 3.09]	0 [25.33 3.09]
1 [20.00 2.27]	10 [24.00 1.28]
3 [14.07 1.48]	20 [22.67 0.00]
10 [6.91 0.67]	40 [20.00 0.00]
30 [2.81 0.26]	

A4. The Bias–Variance Experiment

```

1 TRUE, SIG, REPS = lambda a: 30 + 34*a, 6.0, 500
2 grid = np.linspace(0.8, 2.4, 17); gu = (grid - 1.6)/0.8
3 design = lambda uu, d: np.column_stack([np.ones_like(uu)] +
4                                         [uu**k for k in range(1, d+1)])
5
6 def experiment(degree, lam=0.0):
7     r = np.random.default_rng(99)
8     A, Ag = design(u, degree), design(gu, degree)
9     D = np.eye(degree+1); D[0, 0] = 0      # never penalise the intercept
10    curves = np.zeros((REPS, len(grid)))
11    for s in range(REPS):
12        yy = TRUE(x) + r.normal(0, SIG, 9)    # a FRESH survey every time
13        c = np.linalg.solve(A.T@A + max(lam, 1e-12)*D, A.T@yy)
14        curves[s] = Ag @ c
15    bias2 = np.mean((curves.mean(0) - TRUE(grid))**2)    # where the cloud sits
16    var = np.mean(curves.var(0))    # how wide it is
17    return round(bias2, 2), round(var, 2), SIG**2, round(bias2+var+SIG**2, 2)
18
19 for deg in [1, 2, 4, 8]: print("degree", deg, experiment(deg))
20 for lam in [0, 0.01, 0.1, 0.3, 1, 3, 10]:
21     print("degree 8, lam =", lam, experiment(8, lam))

```

A5. The Pipeline

```

1 from sklearn.pipeline      import make_pipeline
2 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
3 from sklearn.linear_model  import RidgeCV, LassoCV, ElasticNetCV
4 from sklearn.model_selection import cross_val_score
5
6 alphas = np.logspace(-3, 2, 30)
7 X, Xt = x.reshape(-1, 1), xt.reshape(-1, 1)      # raw area, nothing else
8
9 models = {
10     "ridge": make_pipeline(PolynomialFeatures(8), StandardScaler(),
11                           RidgeCV(alphas=alphas)),
12     "lasso": make_pipeline(PolynomialFeatures(8), StandardScaler(),
13                           LassoCV(alphas=alphas, cv=3, max_iter=500000)),
14     "enet" : make_pipeline(PolynomialFeatures(8), StandardScaler(),
15                           ElasticNetCV(alphas=alphas, cv=3, max_iter=500000,
16                                         l1_ratio=[.1,.5,.7,.9,.95,.99,1])),
17 }
18 for name, pipe in models.items():
19     pipe.fit(X, y)      # the powers AND the scaling live inside
20     print(f"{name:6s} alpha*={pipe[-1].alpha_:8.4f}  test={rmse(pipe,Xt,yt):6.2f}")

```